TwinCAT Boot + HMI Server update scripts

Where	Run	What it does
Bench system	`Create-UpdatePackage.bat`	Uses ADS to switch TwinCAT to Config mode and stops the HMI Server (`TcHmiSrv`). Copies the Boot folder and HMI Server folder to a local build folder with a SHA-256 manifest, restarts both, then zips the package to a single file, `<USB>:\TwinCAT_Update.zip`.
Machine IPC	`Apply-Update.bat` (started by the PLC)	Finds the zip on the USB stick, unzips it to a local staging folder and verifies it. Uses ADS to switch to Config mode and stops the HMI Server. Backs up the old version, replaces the folders, verifies them, rolls back if anything fails, then reboots.

```
Apply-Update.bat / Create-UpdatePackage.bat   launchers
Config.ps1                                    paths, service name, options  <-- check this first
scripts\Common.ps1                            shared functions
scripts\New-UpdatePackage.ps1                 bench logic
scripts\Install-UpdatePackage.ps1             machine logic
UpdateScreen\                                 optional "update in progress" screen (see below)
```

One-time setup

1. **Check the paths in `Config.ps1`** against a real IPC. The first folder that exists is used, and the log shows which one.
   - Boot: `C:\ProgramData\Beckhoff\TwinCAT\3.1\Boot` (4026) or `C:\TwinCAT\3.1\Boot` (4024)
   - HMI: `C:\ProgramData\Beckhoff\TF2000 TwinCAT 3 HMI Server` or `C:\TwinCAT\Functions\TF2000-HMI-Server\service`
2. **No extra software is needed for ADS.** The scripts use the .NET ADS API (`TwinCAT.Ads.dll`) that comes with TwinCAT 3. A self-contained v4.x copy is bundled next to `Config.ps1` and is tried first. If it doesn't work, they try the GAC, then search the TwinCAT install folders, preferring v4.x DLLs. On the machine, the DLL is found and connected before anything is stopped. If it can't be found, the update stops and nothing is changed. To check an IPC, especially a runtime-only (XAR) image:

   ```
   Get-ChildItem 'C:\TwinCAT','C:\Program Files (x86)\Beckhoff' -Recurse -Filter TwinCAT.Ads.dll -EA 0
   | Select FullName
   ```
3. **On each machine**, copy this whole folder to a fixed local path, for example `C:\TwinCAT_UpdateTool\`. The PLC needs a path that doesn't depend on the USB drive letter. The USB stick only needs to hold `TwinCAT_Update.zip`.

PLC trigger (Tc2_Utilities `NT_StartProcess`)

```
VAR
    fbStartUpdate : NT_StartProcess;
    bHmiUpdateButton : BOOL;   // linked to the HMI button
END_VAR

fbStartUpdate(
    NETID     := '',
    PATHSTR   := 'C:\Windows\System32\cmd.exe',
    DIRNAME   := 'C:\TwinCAT_UpdateTool',
    COMNDLINE := '/c C:\TwinCAT_UpdateTool\Apply-Update.bat',
    START     := bHmiUpdateButton,   // rising edge starts it
    TMOUT     := T#5S);
```

TwinCAT keeps running until the package has been fully validated. If the stick is missing or the package is bad, nothing is stopped and the machine keeps running. Once validation passes, the PLC and HMI go down and the IPC reboots about 10 s after the update finishes.

Update status screen (optional)

On a machine with an HMI panel, `UpdateScreen\` switches the **TF1200 UI Client** from the HMI to a status page while Apply-Update runs. The page says the system is being updated, warns not to switch off the power or remove the USB stick, and shows live progress: a progress bar, the current step, file counts and elapsed time. At the end it shows the result: *Update complete* (with the restart countdown), *Update not applied* (nothing changed), *previous version restored*, or *service needed* if the rollback failed.

```
UpdateScreen\Install-UpdateScreen.bat   run once per machine (admin): registers the scheduled task
UpdateScreen\Show-UpdateScreen.ps1      switches the TF1200 UI Client to the page (or opens a fallback
window)
UpdateScreen\UpdateScreen.html          the page itself: edit text/colours/logo here
```

Setup: after copying the tool to `C:\TwinCAT_UpdateTool\`, run `UpdateScreen\Install-UpdateScreen.bat` once. To remove it: `Install-UpdateScreen.bat -Uninstall`. Nothing in the UI Client's own configuration has to be changed.

Preview without running an update, with the UI Client open (simulated progress; it switches to the page and ends back on the HMI after about 2.5 minutes). `-DemoResult` can be `Success`, `Failed`, `RolledBack` or `RollbackFailed`:

```
powershell -STA -ExecutionPolicy Bypass -File UpdateScreen\Show-UpdateScreen.ps1 -Demo -DemoResult Success
```

How it works

- **Switching the UI Client.** The UI Client has no way to change its page from outside while it runs: it loads `startUrl` from its configuration file once at startup. So when an update starts, `Show-UpdateScreen.ps1` reads the configuration the running UI Client uses (its `--config-path`, or `%AppData%\Beckhoff\TF1200-UI-Client\config.json`). It writes a copy with only `startUrl` changed to `%TEMP%\TwinCAT_UpdateScreen\`, and restarts the UI Client with `--config-path` pointing at that copy. Kiosk mode, keys and all other settings stay the same. **The real configuration file is never changed**, so after the restart at the end of the update the UI Client opens the HMI as usual. If the UI Client does not come up on the page within 20 s, it is started again the way it was.
- **When the update ends without a restart** (*Update not applied*, or a run with `-NoReboot`), the page goes back to the UI Client's normal start page by itself: 20 s after success, 120 s after a failure, or straight away with the *Close* button. *Service needed* stays until someone presses *Close*. During the restart countdown there is no Close button.
- **The HMI stays up if nothing happens.** The UI Client is only switched once Apply-Update has really started (within 90 s of the button press). If the stick is missing, the page shows *Update not applied* with the reason.
- **Fallback:** if no UI Client is running in the logged-on session, a full-screen PowerShell (WPF) window shows the same page instead. It can't be closed with Alt+F4 while the update runs. For maintenance, **Ctrl+Shift+F12** always closes it. That window uses the Internet Explorer 11 engine, so keep any edits to `UpdateScreen.html` IE11-compatible (no CSS variables, no `let` / `const` /arrow functions).
- **The update never depends on the screen.** `Install-UpdatePackage.ps1` only writes its progress to `C:\TwinCAT_UpdateBackups\update_progress.json` and `update_progress.js` (the page in the UI Client loads the `.js` copy, because a local page in Chromium can't read JSON files). Any error writing them is ignored. If the screen is not installed or fails, the update runs exactly the same.
- **Why a scheduled task:** the PLC starts Apply-Update as SYSTEM in session 0, where windows are invisible and the UI Client of the logged-on user can't be reached. So `Apply-Update.bat` runs `schtasks /Run /TN "TwinCAT Update Screen"` and ignores the result. The task runs as the local *Users* group, so it runs in the session of whoever is logged on (e.g. the HMI auto-logon user). With no one logged on, nothing is switched and the update still runs.
- Errors from the PowerShell window are logged to `%TEMP%\TwinCAT_UpdateScreen.log` of the logged-on user. The logged-on user needs read access to `C:\TwinCAT_UpdateBackups` (the default permissions under `C:\` allow this).

Restart after the update

After a successful update (and after a rollback) the IPC does a full restart (`shutdown /r /f`, never a Fast Startup / hybrid boot), `RebootDelaySeconds` (10 s) after the result is shown, so TwinCAT and the HMI start cleanly on the new Boot folder. Running programs are closed without asking. If Windows refuses to schedule the restart, Apply-Update logs the reason and restarts with `Restart-Computer -Force` instead. A rollback that failed never restarts: the IPC stays in Config mode for service. `RebootAfterApply = $false` in `Config.ps1`, or `-NoReboot`, turns the restart off.

Optional arguments

Both launchers pass anything after them straight to their script, e.g. `Apply-Update.bat -NoReboot` or `Create-UpdatePackage.bat -Destination E:\`. The PLC trigger runs `Apply-Update.bat` with no arguments.

`Apply-Update.bat` (machine)

Argument	What it does
`-AllowVersionMismatch`	Applies the package even if the XAR / TF2000 versions differ from this machine. The mismatch is still logged. Manual use only.
`-NoReboot`	Doesn't reboot at the end. Restarts the HMI Server and puts TwinCAT back into Run mode instead.
`-PackagePath <zip or folder>`	Uses this package zip (or the folder holding it) instead of searching all drives for `\TwinCAT_Update.zip`. Needed if more than one drive holds a package. An unzipped package folder from an older version of the scripts also works.
`-ConfigPath <file>`	Uses a different config file instead of `Config.ps1`.

`Create-UpdatePackage.bat` (bench)

Argument	What it does
`-Destination <drive or folder>`	Where to build the package, e.g. `E:\`. Default: the USB drive the script runs from, or the only USB drive present.
`-NoStop`	Copies without stopping TwinCAT / the HMI Server first. Not recommended: files can change mid-copy.
`-LeaveStopped`	Leaves TwinCAT in Config mode and the HMI Server stopped afterwards.
`-ConfigPath <file>`	Uses a different config file instead of `Config.ps1`.

Set the environment variable `NOPAUSE=1` to stop `Create-UpdatePackage.bat` waiting for a keypress at the end (useful when calling it from another script).

Result and logs (machine)

- `C:\TwinCAT_UpdateBackups\last_update_status.txt`: `SUCCESS` / `FAILED - nothing changed` / `FAILED - previous version restored` / `FAILED - ROLLBACK FAILED`
- `C:\TwinCAT_UpdateBackups\Logs\Apply_<time>.log`. A copy also goes to `<USB>:\TwinCAT_Update_Logs\`.
- `C:\TwinCAT_UpdateBackups\<time>_before_update\`: the old Boot and HMI folders. The last 5 are kept (`BackupsToKeep`).

Exit codes: 0 ok, 1 failed (nothing changed), 2 failed and rolled back, 3 rollback failed (the IPC is left in Config mode and **not** rebooted), 4 already running.

Manual rollback to an old version: copy `<time>_before_update\Boot` and `\HMI` back to their folders while TwinCAT is in Config mode and `TcHmiSrv` is stopped, then reboot. Or put that folder on a stick as `TwinCAT_Update`, set `"Purpose": "UpdatePackage"` in its `manifest.json`, and run Apply-Update. Each backup has its own `versions.json`, so the version check still applies.

Important behaviour

- **PLC persistent data is kept.** `PreservePersistentData = $true` means `*.bootdata` files are neither packaged nor overwritten, so each machine keeps its own retained values. If the new PLC software changes persistent variables, check how TwinCAT handles that before you rely on it.
- **Mirror copy.** The target folders end up exactly like the package. Stale files are removed, except excluded files, which are left alone.
- **Version check (exact match).** The bench writes `versions.json` into the package. It records the TwinCAT XAR version (registry `TcVersion`, e.g. `3.1.4026.27`) and the TF2000 HMI Server version (ProductVersion of the exe the `TcHmiSrv` service runs, e.g. `25.0.122`). Apply-Update reads the same values on the machine. If either differs, or can't be read, it stops with an error **before anything is stopped or changed**, and both versions are written to the log. The bench refuses to build a package if it can't read either version. Running `Apply-Update.bat -AllowVersionMismatch` by hand overrides the check; the PLC trigger never does.
- **Hardware binding.** The Boot folder holds the bench's I/O configuration. EtherCAT adapters are bound to a MAC address or PCI slot, so the bench and machines should be the same IPC model and use the same adapter binding. Licences are not part of the Boot folder and don't move with it.
- **Zipped package.** The package is one file at the root of the stick, `TwinCAT_Update.zip` (Boot, HMI, `manifest.json`, `versions.json` inside). Next to it are the bench log `TwinCAT_Update_create.log` and, once a machine has run the update, the `TwinCAT_Update_Logs\` folder. The bench builds the package in `%TEMP%\TwinCAT_Update_build` and restarts TwinCAT and the HMI Server *before* zipping, so the downtime is only the copy. The machine unzips to `C:\TwinCAT_UpdateBackups\Staging`, checks every file's SHA-256 there, and deletes the staging folder at the end. Needs .NET Framework 4.5 or newer (part of every TwinCAT 3 install).
- Incomplete zips are written as `TwinCAT_Update.zip.partial` and are never picked up. The previous package on the stick is kept as `TwinCAT_Update_previous.zip` (and its log as `TwinCAT_Update_create_previous.log`). A `TwinCAT_Update` folder from an older version of the scripts is renamed to `TwinCAT_Update_old` the first time the new version builds a package (it can be deleted). Apply-Update still accepts packages in the older layouts.